move from one element to the next by just incrementing the memory address by the size of the elements.

Moreover, being allocated in a single block of memory, arrays are more likely to show the so-called locality of reference: when traversing an array, for example, data in the same page of memory are likely to be accessed in a short window of time. This can lead to several optimizations (see the original article about the locality principle).

The key advantage of arrays is that they have constant time access for all elements. It would be more accurate to rather say that every *position* in an array can be accessed in constant time: it's possible to retrieve or store the first element, or the last, or any intermediate element as long as its position is known: see figure C.2 as a reference.

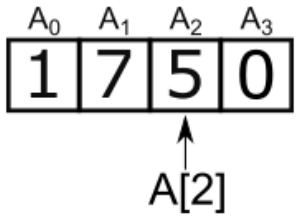


Figure C.2 Retrieving the third element of an array. If the array is stored in a variable named A, then A[2] refers to the third element of the array. This is because on computers memory addresses, and thus array elements' indices, starts from 0.

While random access is one of the strengths of arrays, other operations are slower for them. As mentioned, you can't resize an array by just adding or removing an element at its tail. Every time such an operation is needed, you have to reallocate the array. Unless you are using a dynamic array: in that case you have to initially allocate a larger chunk of memory, keep track of the number of elements, and amortize the overhead for resizing on a larger number of operations.